

Cross-Field Translation in Automated Theorem Proving

A Translation Graph Framework for Mathematical Reasoning

Experimental findings from a LeanDojo-based theorem proving system trained on Mathlib4.

1. The Core Insight

When mathematicians solve hard problems, they often succeed not by applying stronger tools within the problem's native domain, but by **translating the problem into a different mathematical language** where existing tools are more powerful. Gauss's multiple proofs of quadratic reciprocity — via counting, analysis, cyclotomic fields, and geometry — demonstrate that the same theorem can be proved through fundamentally different mathematical worlds. The Fundamental Theorem of Algebra admits ~100 proofs spanning topology, complex analysis, and Galois theory.

We propose that this **cross-field translation ability** is the key capability an AI theorem prover must learn. Rather than searching for the next tactic within a fixed domain, the prover should ask: *What mathematical world should I translate this problem into?*

2. The Translation Graph

We formalize this insight as a **Translation Graph**: a directed weighted graph where nodes are mathematical domains and edges are tactics that translate between them. Each edge carries a learned success rate from proof data.

2.1 Domain Nodes

Domain	Description	Theorems Proved
Set	Set theory: $\cup, \cap, \subseteq, \emptyset$, univ	78
Finset	Finite sets: insert, card, disjoint	133
Nat	Natural numbers: +, *, mod, div	51
Logic	Propositional: $\vee, \wedge, \neg, \leftrightarrow$	(bridge domain)
Arithmetic	Solvers: omega, ring, linarith	(bridge domain)
Membership	Element-level: $x \in S, \forall x$	(bridge domain)

2.2 Translation Edges (Cross-Field)

The most important edges are **cross-field translations** — where the solution domain differs from the problem domain.

Translation	Count	Rate	Key Tactic	Insight
Nat → Arithmetic	47	47%	omega, ring	Number theory → algebraic solvers
Set → Logic	15	100%	simp [Set.ext_iff]	Set equality → $\forall x, \leftrightarrow$
Finset → Set	12	100%	simp [Set.union_self]	Finite set → general set tools
Set → Membership	2	100%	simp [Set.mem_union]	Whole-set → element-level
Finset → Logic	0	0%	(missing)	61 unproved theorems here

3. Key Finding: 35% of Proofs Use Cross-Field Translation

Of 262 proved theorems, **93 (35%)** were proved by translating the problem to a different mathematical domain. The most powerful translation is **Nat → Arithmetic** (47 theorems): instead of structural induction on natural numbers, the model uses algebraic solvers like *omega* and *ring* that operate in a completely different mathematical framework.

The second most impactful translation is **Set → Logic** (15 theorems): reducing set equality to pointwise propositional logic via *Set.ext_iff*. This is exactly the kind of 'field translation' that human mathematicians perform instinctively — recognizing that $s \cup t = t \cup s$ is really $\forall x, (x \in s \vee x \in t) \leftrightarrow (x \in t \vee x \in s)$, which is trivially true by commutativity of $\vee$.

4. The Missing Edges: Where the Model Fails

115 theorems remain unproved after search. 61 of these are Finset theorems. The translation graph reveals **why**: the edge **Finset → Logic** has a 0% success rate. The model cannot translate Finset problems into propositional logic the way it translates Set problems.

This is not because the translation is impossible — *Finset.mem_insert* is fundamentally just $a \vee b = x \vee a \in s$. It is because the model lacks the **bridge tactics** that connect Finset to logic. The **Set → Logic** bridge (*ext_iff*, *subset_def*) has no Finset counterpart in our action space. This is a concrete, actionable gap.

5. Algorithmic Representation

The translation graph is not just an analytical tool — it is a **computable data structure** that can guide tactic generation. We implement a three-phase algorithm:

Phase	Operation	Implementation
1. DETECT	Identify the domain(s) of the proof state	Pattern matching on state symbols ($\cup \rightarrow$ Set, $\in \rightarrow$ Membership, $\geq \rightarrow$ Order)
2. PLAN	Rank available translations by success rate	Query graph edges from detected domain, score by confidence $\times$ cross-f
3. ACT	Generate tactics for the chosen translation	Return Lean tactics from the highest-ranked edge

This algorithm can operate as a standalone policy or as a **planning layer** above an existing generative model. In the latter configuration, the translation graph decides *which domain to target*, and the generative model fills in the specific tactic syntax. This separation mirrors how human mathematicians think: the strategic decision ('use topology, not algebra') is distinct from the tactical execution ('apply the intermediate value theorem').

6. Connection to the Broader Vision

The translation graph is a small-scale instance of a much larger principle. The collaborator's insight — that AI should learn to translate problems between mathematical fields, not just search within one — applies at every scale of mathematics:

Scale	Translation Example	Our System
Elementary	Set equality $\rightarrow$ propositional logic (ext_iff)	Working: 15 theorems proved
Undergraduate	Group theory $\rightarrow$ linear algebra (representation theory)	Future: requires larger Mathlib coverage
Graduate	Number theory $\rightarrow$ algebraic geometry (schemes)	Future: requires deep theory imports
Research	Automorphic forms $\rightarrow$ Galois representations (Langlands)	Long-term: the 'holy grail' of math AI

The architecture is the same at every level. What changes is the size of the domain vocabulary and the depth of the translation chains. Our system's Set $\rightarrow$ Logic translation is structurally identical to Grothendieck's Number Theory $\rightarrow$ Algebraic Geometry translation — both recognize that a problem stated in language A becomes trivial when restated in language B.

7. Experimental Results Summary

Model	Parameters	Training Data	Proved	Rate
v1-v3 (T5-small)	60M	7.6K traces	19-22 / 31	61-71%
v5 (T5-small)	60M	68K traces	200 / 555	36%
v6 (T5-base)	220M	195K traces	262 / 554	47%
Premise-augmented	60M	68K + premises	19 / 30	63%*

*Premise augmentation underperformed at 60M params (63% vs 83% baseline on same set). This confirms that retrieval-augmented generation requires sufficient model capacity, consistent with the ReProver finding that 299M parameters are needed.

8. Next Steps

Immediate: Add the missing Finset $\rightarrow$ Logic bridge tactics to the action space. The translation graph identifies exactly which edges are missing and what tactics would implement them.

Medium-term: Train the generative model to predict translation edges directly. Instead of generating a tactic, the model first predicts 'translate to logic,' then generates the implementing tactic. This two-phase architecture separates strategic reasoning from tactical execution.

Long-term: Scale the translation graph to cover all of Mathlib. With 200K+ theorems providing training signal, the graph becomes a map of how mathematical fields connect — a computational version of the Langlands Program's vision of unified mathematics.